# Multithreaded Railway Reservation System

Java-Based Console Application

Name : ARIJIT DEB
Stream : IT
Sec : A
Roll No: 75

# Introduction & Key Features

## What is a Reservation System?

A reservation system manages bookings for limited resources, ensuring fair allocation and preventing conflicts when multiple users attempt to book simultaneously.

**Concurrency Importance:** Real-world booking systems handle thousands of users simultaneously, requiring thread-safe operations to prevent double bookings.

**Project Objective:** Simulate seat booking using multithreading in Java to demonstrate concurrent programming concepts.

## Core Features

- Seat booking system with seat allocation
- Seat cancellation with ticket refunds
- Real-time view of available seats
- Multithreading support for concurrent bookings
- File handling to store booking records

# System Architecture

The system follows a layered architecture where each component has a specific responsibility, working together to provide a complete booking solution.

### User Interface

Menu-driven console input via Scanner

### ReservationSystem

Core booking logic and seat management

### Ticket

Booking confirmation

### File Storage

Data is stored in bookings.txt

# Core Classes

## Passenger

**Data Model:** Stores passenger details including name and ID

- Constructor for initialization
- Getter methods for data access

## Ticket

**Booking Details:** Manages ticket information

- Ticket ID entered by user
- Seat number tracking
- Passenger allocation
- toString() for display

## ReservationSystem

**Core Logic:** Thread-safe seat management

- HashMap for booked seats
- ArrayList for available seats
- Synchronized methods
- Booking conflict prevention

# Thread Management & Data Persistence

## BookingThread Class

**Concurrency Implementation:** Each booking runs in a separate thread, simulating multiple users attempting to book seats simultaneously.

- Implements Runnable interface
- Thread.sleep() for simulation delay
- Independent booking execution
- Thread-safe seat allocation

## FileHandler Class

**Persistent Storage:** Writes booking data to bookings.txt using buffered writers for efficient file operations.

- BufferedWriter for performance
- FileWriter for data persistence
- Exception handling for I/O errors
- Appends new bookings to file

# Working Example

### User Input

Passenger enters name, ID, and desired seat number through console menu

### Thread Start

BookingThread starts execution in separate thread, simulating concurrent booking attempt

### Seat Allocation

ReservationSystem checks availability and allocates seat if available

### Ticket Generation

Ticket object created with unique ID, seat number, and passenger details

### Data Storage

FileHandler writes booking details to bookings.txt for persistent record

# Synchronization Concept

## Why synchronized?

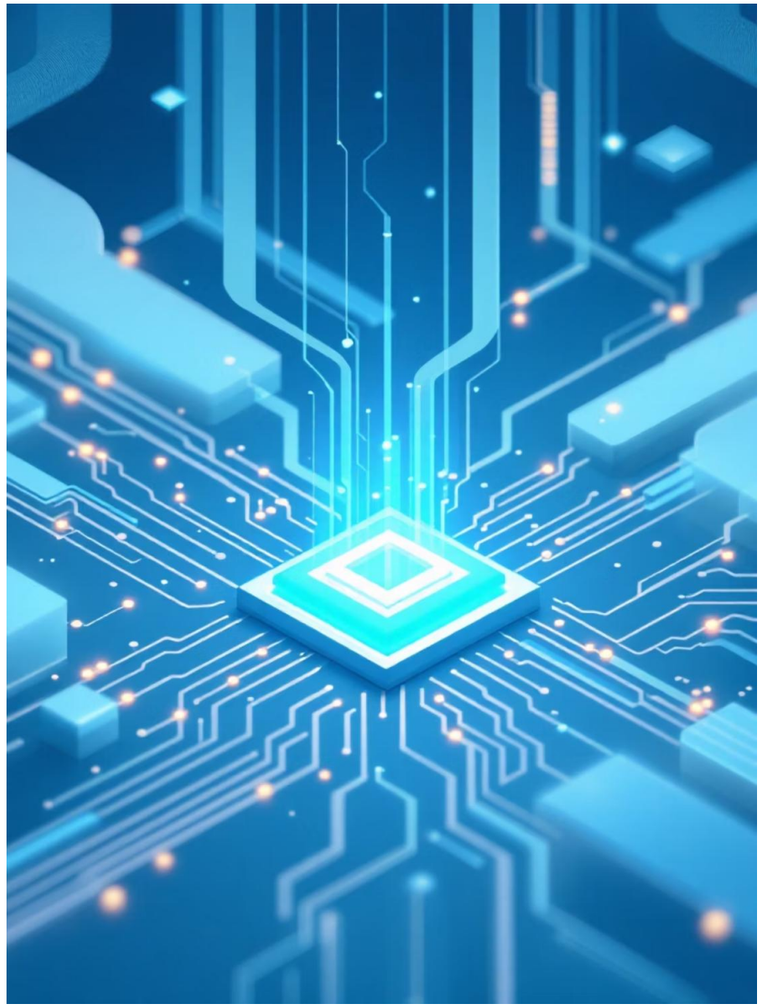
The **synchronized** keyword ensures thread safety by preventing race conditions when multiple threads attempt to book the same seat simultaneously.

## How It Works

- Only one thread can execute synchronized methods at a time
- Prevents double booking scenarios
- Ensures data consistency across threads
- Uses intrinsic locks (monitors)

## Without Synchronization

Multiple threads could simultaneously check seat availability and both book the same seat, leading to overbooking.



# Advantages & Limitations

## Advantages

- **Thread-Safe Design**

  Synchronized methods prevent race conditions and ensure data integrity

- **Simple & Efficient**

  Lightweight implementation using core Java features without external dependencies

- **Real-World Simulation**

  Accurately models concurrent booking scenarios found in actual reservation systems

## Limitations

- **Console-Based UI**

  Limited to text-based interface, less user-friendly than GUI applications

- **File Storage Only**

  Uses flat files instead of relational database, limiting query capabilities

- **Scalability Issues**

  File-based approach doesn't scale well with thousands of concurrent users

# Future Enhancements

### Graphical User Interface

Implement JavaFX or Swing for intuitive, modern interface with visual seat selection and real-time updates

### Database Integration

Connect to MySQL or PostgreSQL for robust data management, complex queries, and better scalability

### Online Booking System

Add network capabilities using sockets or web services for distributed multi-user access

### Real-Time Tracking

Implement WebSocket or similar technology for live seat availability updates across all users

# Conclusion

## Project Summary

This multithreaded railway reservation system demonstrates practical application of concurrent programming in Java. The implementation successfully simulates real-world booking scenarios where multiple users compete for limited resources.

## Key Learning Points

- Thread synchronization prevents race conditions
- Proper data structures manage state efficiently
- Persistent storage ensures data durability
- Modular design promotes maintainability

**Multithreading is essential** for building scalable, responsive systems that handle concurrent access in real-world applications.